5. ROS 2 Package

1. Package Introduction

Each robot may have many functions, such as motion control, visual perception, and autonomous navigation. If the source code of these functions is all put together, that is of course also possible, but when we want to share some of these functions with others, we will find that the code is all mixed together, and it is very difficult to separate them out.

Packages work on this principle: the code of different functions is divided into different packages to minimize the coupling between them. When a package needs to be shared with others in the ROS community, only how to use this package needs to be explained, and others can quickly start using it.

Therefore, the package mechanism is one of the important methods for improving the software reuse rate in ROS.

2. Create a Package

How to create a package in ROS 2? This command can be used:

```
ros2 pkg create <package_name> --build-type <build-type> --dependencies
<dependencies> --node-name <node-name>
```

In the ros2 command:

- **pkg**: indicates functions related to packages;
- **create**: indicates creating a package;
- **package_name**: required: the name of the new package;
- **build-type**: required: indicates whether the newly created package is C++ or Python. If C++ or C is used, use ament_cmake here; if Python is used, use ament_python;
- **dependencies**: optional: indicates the dependencies of the package. A C++ package needs to include rclcpp, a Python package needs to include rclpy, plus other required dependencies;
- **node-name**: optional: the name of the executable program; the corresponding source file is automatically generated and a configuration file is generated;

For example, create C++ and Python versions of the package in the terminal respectively:

- Switch to the src directory of the workspace
- Replace workspace here with your actual folder path

```
cd workspace/src
```

- Create a C++ package example

```
ros2 pkg create pkg_helloworld_cpp --build-type ament_cmake --dependencies rclcpp
--node-name helloworld
```

- Create a Python package example

```
ros2 pkg create pkg_helloworld_py --build-type ament_python --dependencies rclpy
--node-name helloworld
```

3. Compile the Package

In the created package, the code can continue to be written; afterwards, compilation and environment variable configuration are required before it can run normally:

- Switch to the workspace directory
- Compile all packages

```
colcon build
```

- Compile a specified package

```
colcon build --packages-select pkg1 pkg2
```

4. Complete Workspace Structure with Packages

The directory structure of a ROS 2 workspace is as follows:

```
WorkSpace --- User-defined workspace.
    |--- build: directory for storing intermediate files; a separate subdirectory
is created under this directory for each package.
    |--- install: installation directory; a separate subdirectory is created
under this directory for each package.
    |--- log: log directory, used to store log files.
    |--- src: directory for storing package source code.
        |-- C++ package
            |-- package.xml: package information, such as package name, version,
author, and dependencies.
            |-- CMakeLists.txt: configures build rules, such as source files,
dependencies, and target files.
            |-- src: C++ source file directory.
            |-- include: header file directory.
            |-- msg: message interface file directory.
            |-- srv: service interface file directory.
            |-- action: action interface file directory.
        |-- Python package
            |-- package.xml: package information, such as package name, version,
author, and dependencies.
            |-- setup.py: similar to CMakeLists.txt of a C++ package.
            |-- setup.cfg: basic package configuration file.
            |-- resource: resource directory.
            |-- test: stores test-related files.
            |-- directory with the same name as the package: Python source file
directory.
```

In addition, both Python and C++ packages can customize some directories related to configuration files.

```
|-- C++ or Python package
    |-- launch: stores launch files.
    |-- rviz: stores rviz2 configuration-related files.
    |-- urdf: stores robot modeling files.
    |-- params: stores parameter files.
    |-- world: stores simulation environment-related files.
    |-- map: stores map files required for navigation.
    |-- ......
```

The above directories can also be defined with other names, or some other directories can be created as needed.